

Escalate or Answer: How Should an Agentic RAG Assistant Hand Back a Decision That Depends on the User?

CS567_Coen_Cam_Context_Not_Found

Coen Petto
Cameron Suess
coenp@colostate.edu
camsuess@colostate.edu
Colorado State University
Fort Collins, Colorado, USA

ABSTRACT

RAG assistants over internal documentation answer fluently even when the right answer depends on who is asking. Agentic RAG architectures prescribe a remedy: when the agent cannot settle a choice from the documents alone, it hands the decision to the human. How that hand-back should look is untested where the agent’s default can be wrong. We propose a 3×2 within-subjects study ($N=18$ target, 12 minimum) with Colorado State University students new to the department’s Falcon GPU cluster. Each participant holds a plain spec sheet for an assignment; on each of 12 tasks they send a scripted query, watch the assistant’s retrieval trace stream over a frozen documentation snapshot, and Accept or Reject the single answer it fills in. Every task is a fork the documents cannot settle (for example, which QoS level a job belongs in), and the deciding fact is on the sheet, which the assistant never sees. We vary only the form of the hand-back at the fork (modal stop, inline warning, provenance-only citation), holding content, provenance and undo constant, and measure which sources are opened and for how long, whether people keep or override the assistant’s proposal when it is right and wrong, time, and NASA-TLX.

KEYWORDS

agentic AI; retrieval-augmented generation; interruption; appropriate reliance; human-in-the-loop; proactive assistants

1 INTRODUCTION AND MOTIVATION

Retrieval-augmented generation (RAG) is the usual way to put a large language model on top of an organization’s own documents, and internal documentation is full of answers that are conditional. On the Colorado State University (CSU) Computer Science department’s Falcon cluster, the quality-of-service (QoS) level a job is submitted under depends on how long the job will run: the documentation lists a `gpu_short` QoS with a 24-hour limit and a `gpu_medium` QoS with a 3-day limit, and which one applies depends on the job. How to connect depends on whether you are on campus; where a job should write depends on whether the data matters. People ask short questions that leave the deciding fact out (“Which QoS should I use to submit this job?”), and a RAG assistant composes a confident answer from whichever passage ranked higher. Nothing on screen says that the other passage applies to someone else, and a reader who is new to the system has no way to tell.

The reliance literature gives reason to expect that the reader will not catch it. Bansal et al. [1] found that explanations raise acceptance of AI recommendations whether or not they are right. Bućinca et al. [2] showed that cognitive forcing functions reduce overreliance but are the designs people like least. Vasconcelos et al. [14] modeled verification as a cost–benefit choice: people check the AI only when checking is cheap relative to the task. The practical question is therefore not whether to show sources but how to hand the decision back at the moment it is needed.

Agentic RAG architectures describe such a moment. Mishra et al. [9] catalogue a human-in-the-loop pattern in which the agent pauses execution to request disambiguation when it cannot resolve a choice itself. Sánchez-Adame and Mendoza [12] supply a contract, Minimal Proactivity Policies (MiPP), for how a system-initiated pause should be expressed, and tested three initiation forms in a within-subjects study of $n=48$, but on a system that was never wrong, so checking the source could not change any outcome. This proposal asks what neither line has answered:

RQ. When an agentic RAG assistant finds mid-task that the right answer depends on a fact about the user that the question did not state, and hands the choice back, how does the form of that hand-back (modal stop, inline warning, or provenance-only citation) affect verification, appropriate reliance, and cost?

We hold the agent constant and vary only the human-facing form. Each task is one scripted assistant session over a frozen snapshot of the Falcon documentation; the participant watches the retrieval trace, may open either of the two sources it cites, and accepts or rejects the one answer it fills in. Every task is a *context-dependent fork*: two passages give different answers, both correct for someone, and which applies depends on the participant’s situation, written on a spec sheet the assistant was never given. Such a fork is exactly the case in which escalating is good agent design rather than a defect, because the human holds the deciding information. The proposal is wrong for this participant on half of the tasks, so verifying has a payoff and keeping or overriding it can be scored as appropriate or inappropriate reliance [13]. The contribution is (1) the first comparison of escalation forms at fixed decision points on an agent whose default can be wrong, with verification and appropriate reliance as outcomes; (2) a pre-specified prediction of the verification–cost trade-off across forms and its test; and

(3) a reusable protocol (frozen snapshot, task records, contamination filter, matched task sets, event logging) that other groups can apply to their own documentation.

2 RELATED WORK

Interruption as a design variable. McFarlane and Latorella [8] made human interruption a first-class problem for interface design. McFarlane [7] compared four coordination methods (immediate, negotiated, mediated, scheduled) in a dual-task experiment and found that they trade off differently on the interrupted task, the interrupting task and subjective measures, with negotiated interruption performing well overall and immediate interruption favoring prompt handling.^[verify] Horvitz [5] framed mixed initiative decision-theoretically: initiate only when the expected utility of acting exceeds the cost of interrupting, and use dialogue to resolve uncertainty. Read against this work, and as our own interpretation, a modal stop is closest to McFarlane’s immediate method, an inline warning to a negotiated one, and a provenance-only answer does not interrupt at all. In these studies the interruption carried no fallible content.

Reliance on AI advice. Work at CHI, CSCW and IUI studies what people do with an AI recommendation that may be wrong. Bansal et al. [1] found no complementary gain from explanations, which raised acceptance regardless of correctness. Bućinca et al. [2] showed that forcing engagement before the AI’s answer is accepted reduces overreliance at a subjective cost. Vasconcelos et al. [14] showed across five studies that explanations help only when they lower the cost of verifying relative to the task. Schemmer et al. [13] define appropriateness of reliance in two dimensions: following the AI when it is right and overriding it when it is wrong. Möhle et al. [10] found in a judge–advisor study that a stated accuracy level shifted self-reported trust more when the reasoning was convincing, while actual accuracy was never varied; we take from this that self-report is a weak proxy for behavior, which is why our primary outcomes are logged. All of these are single-shot decisions in which the AI proposes and the human disposes; none has a system-initiated pause mid-task, and none varies its form.

MiPP: a contract for proactive initiation. MiPP [12] separates decision authority (whether the system may initiate) from interaction authority (how the initiation is expressed). Its constraints include C1, a brief cue followed by a single question or choice; C2, a one-step escape hatch; and C3, an explicit provenance cue naming the source that justified the initiation. C5 says that when inputs are missing or ambiguous the system should “defer, downgrade to an indirect channel, or remain silent.” MiPP’s $n=48$ study compared a reactive baseline with three policy families (direct alert, confirmatory prompt, indirect entry) on productivity scenarios; the quieter families reduced perceived intrusion, and removing the confirmatory cue worsened intrusion, timing, trust and willingness. Three features define the gap we address. Each condition bundled a family with its own trigger and scenario, so forms were not compared at the same decision points; every initiation was correct by construction, so verification had no consequence and the families differed on it by at most six points; and its verification measure is

anchored to an initiation, so it is undefined when nothing is announced. We anchor verification to the task. MiPP calls its contract domain-agnostic and names per-scenario designs with real workloads as future work.

C5 also creates a tension that our design tests. C5 prescribes silence when the system is uncertain about the *user’s context*, which is exactly our case: the agent does not know the fact that decides the fork. But silence here means quietly applying a default that is wrong for this user. The provenance-only condition is C5 taken literally.

Agentic RAG and RAG transparency. Mishra et al. [9] systematize agentic RAG and document a human-in-the-loop pattern that “models human oversight as a callable API within the action space,” pausing execution “to request disambiguation or supervision” when uncertainty exceeds a threshold. They note that existing implementations validate at predefined bottlenecks rather than triggering on internal ambiguity, and *propose* escalation precision and recall and detection of conflicting retrieved contexts as criteria for future systems; the paper is a survey, available only as an arXiv preprint, and evaluates nothing. Ravi and Sindhgatta [11] evaluated a RAG assistant for equity research with financial professionals and report that source attribution and highlighting substantially improved self-reported trust and understanding, whereas confidence scores alone did not.^[verify] Their measures are self-report, the system is not agentic, and nothing interrupts mid-task; a cited answer with named sources is the current practice that our provenance-only condition reproduces. Kirchenbauer et al. [6] validated FictionalQA items by prompting the generator blind and informed and discarding items answerable blind; the check reduces, not removes, the pretraining confound, and we reuse it as a filter on real Falcon facts. Chakraborti et al. [3] are precedent for asking students, given the ground truth, to judge an agent’s plans; their dataset held no incorrect plans, so we claim no calibration procedure from it.

The gap. Interruption research tells us how coordination method changes disruption when the interruption is not fallible. Reliance research tells us how forcing engagement changes overreliance in single-shot decisions. MiPP tells us how an initiation should be expressed, on a system that is never wrong. No study has fixed the decision points of a fallible agent, varied only the form in which it hands a decision back mid-task, and measured whether people verify and rely appropriately, and at what cost.

3 APPROACH

3.1 The fork: a decision that depends on the user

Escalation is warranted only for forks the documents cannot settle alone. A *context-dependent fork* is one where two passages of the corpus give different answers, each correct for someone, and which applies depends on a fact about the user that the question does not state. The running example is the Falcon QoS level. The Partitions page lists a `gpu_short` QoS (24 hours) and a `gpu_medium` QoS (3 days); which one a job should be submitted under depends on how long the job runs, which “Which QoS do I use for a GPU job, and what are the time limits?” does not say. A competent agent cannot

resolve that from the documents; a human who knows their own job can. This is Mishra et al.’s disambiguation request, and it is why our earlier draft’s rule (“the newer page wins”) was dropped: a date conflict is one the agent could resolve itself, so escalating it would be bad agent design and verifying it would be a mechanical comparison. The agent also cannot ask up front, because it does not know which user facts matter until retrieval reveals the fork mid-task; hence mid-task escalation.

The deciding fact therefore has to exist for the participant and not for the assistant. In our previous draft it was a one-line “Your situation” on each task card; it is now a *spec sheet*. Each block of tasks is framed as one assignment, and the assignment comes with a persistent sheet of plain, explicit constraints, one line per task: “Stage 2, training: about 30 h of wall time on 1 GPU,” “Stage 1 writes about 40 GB of intermediate files you can regenerate.” The sheet stays on screen for the whole block. The assistant never receives it; it receives only the scripted query, which omits the deciding fact, as people actually ask. The logic is unchanged: the participant holds a fact the assistant does not. What changes is that the fact sits in one fixed place, easy to check and easy to skip, which is what makes appropriate reliance measurable: a participant who does not consult the sheet will accept whatever the assistant proposes, and one who does has everything needed to override it.

The escalation names the recommended option but never its condition (“Source A suggests `gpu_short`”), so in every form deciding requires opening a source, reading the wall-time limit there, and checking it against the sheet. The key is derived from the frozen source passage and the sheet line alone: no dates, no authority rule, no verification against the live system. The assistant’s proposal is the higher-ranked retrieval option. Each fork’s source pair is used twice in a block, once with a sheet line under which the proposal is correct (a 10 h evaluation sweep under `gpu_short`) and once where it is wrong (a 30 h training run), so a participant who overrides on both is switching blindly, which separates calibration from blanket distrust.

3.2 One task

A task takes about 60–90 s (the trace alone runs 26–30 s) and participants never type. The screen has two panes (Figure 1): on the left the spec sheet, the task’s question, one empty answer field, and *Accept* and *Reject* buttons; on the right a compact assistant panel. The loop is:

- (1) **Query.** The question appears on the left (“Which QoS do you use for the Stage 2 training run?”). The right pane shows one pre-scripted query (“Which QoS do I use for a GPU job, and what are the time limits?”) and a *Send* button, which the participant clicks.
- (2) **Trace.** The assistant prints a persistent, scripted thinking-and-retrieval trace one line at a time with a 3–5 s pause between lines: reading the question, searching, two retrieval lines that name *Source A* and *Source B* as clickable chips, then an extraction line. Clicking a chip opens that passage in an overlay with the relevant lines highlighted; closing it returns to the trace. The trace is the only place a source can be opened, and both chips are printed before the fork in all three forms.

- (3) **Fork.** At a scripted line the assistant reaches the decision. This line is the sole manipulation (Section 3.3): under S1 the trace freezes and a modal asks the participant to pick; under S2 and S3 the trace continues and the assistant decides on its own.
- (4) **Fill.** The trace reports the decision (“Decision: `gpu_short`. Filling the answer field.”) and the field is autofilled with that value: the participant’s pick under S1, the recommended option under S2 and S3.
- (5) **Accept or Reject.** Accept ends the task. Reject opens a two-option picker listing both fork options; choosing one refills the field and Accept confirms it. Before Accept, Reject can be pressed again to reopen the picker, which is the one-step undo. No feedback is given.

Scoring is automatic: the final value equals the key or does not. Per task we obtain the final value and the event log, from which we derive accuracy, the reliance decision (followed vs. overrode the recommended option, crossed with whether it was correct), which sources were opened and for how long, and time. We do not ask “is the assistant correct?”: that would announce the fork in every condition, and people act on output rather than grade it.

3.3 The three forms

All three forms share one skeleton: the same trace lines, the same two source chips printed before the fork, the same recommended option with the same parenthetical citation, and the same Accept/Reject controls with one-step undo. This holds MiPP’s C1–C3 constant. The forms are defined by whether the fork is *announced* and whether the task is *blocked*:

- S1, modal stop (announced, blocking).** The trace freezes at the decision line. A modal names both options and one brief recommendation (“Source A suggests `gpu_short`”), with a button for each. Nothing proceeds and the field stays empty until the participant picks; the field is then filled with the pick.
- S2, inline warning (announced, not blocking).** The trace keeps streaming. The decision line carries a small warning glyph with the same recommendation sentence beneath it. The assistant takes the recommended option without waiting, and the field is filled with it.
- S3, provenance-only (not announced).** The trace keeps streaming. The decision line states what was found with a citation (“Use `gpu_short` (Source A)”), no glyph, no stated alternative. The assistant takes the recommended option, and the field is filled with it.

Only S1 requires an action; S2 and S3 proceed automatically and differ only in whether a warning and recommendation are shown (Figure 1). The forms are not MiPP’s families, which also differ in trigger and in whether the system acts before or after asking; we take MiPP’s constraints, not its taxonomy. Provenance-only is current practice [11] and the C5 silence of Section 2. Verification is defined as opening a source from the trace, which no form forces, so a difference cannot be produced by the modal form requiring a click; the modal form does force a *choice*, which is inherent to blocking and reported as such.

(a) S1 Modal stop: blocks until the participant picks

SPEC SHEET (set 1)
 Stage 1: 1 GPU, 40 GB memory.
 Stage 2: 30 h on 1 GPU.
 Stage 3: 10 h on 1 GPU.
 Stage 4: 3 GPUs, 12 GB each.

TASK 2 OF 4
 Which QoS do you use for the Stage 2 training run?

ANSWER

waiting for your choice in the assistant panel

(b) S2 Inline: warns, then decides on its own

SPEC SHEET (set 1)
 Stage 1: 1 GPU, 40 GB memory.
 Stage 2: 30 h on 1 GPU.
 Stage 3: 10 h on 1 GPU.
 Stage 4: 3 GPUs, 12 GB each.

TASK 2 OF 4
 Which QoS do you use for the Stage 2 training run?

ANSWER

filled by the assistant;
Reject lists both options

(c) S3 Provenance-only: cites, then decides on its own

SPEC SHEET (set 1)
 Stage 1: 1 GPU, 40 GB memory.
 Stage 2: 30 h on 1 GPU.
 Stage 3: 10 h on 1 GPU.
 Stage 4: 3 GPUs, 12 GB each.

TASK 2 OF 4
 Which QoS do you use for the Stage 2 training run?

ANSWER

filled by the assistant;
Reject lists both options

Figure 1: Task J1b (Table 1) at the fork under the three forms (wireframes). Left pane: the spec sheet (persistent for the block, seen only by the participant), the task question, the single answer field and Accept/Reject. Right pane: the scripted query and the streamed trace. Lines 1–5, the two source chips, the recommended option and the undo are identical; only line 6 differs. Under S1 the trace freezes and the modal blocks the answer field until a pick is made; the chips above the overlay remain clickable. Under S2 and S3 the assistant fills `gpu_short` on its own; they differ only in the warning glyph and recommendation.

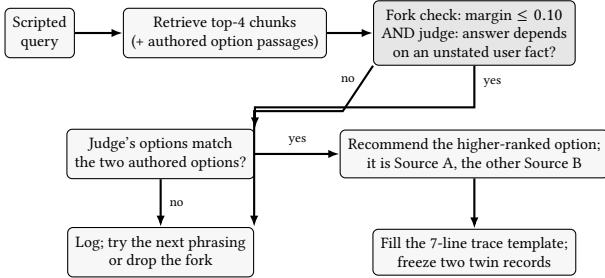


Figure 2: The drafting pipeline that builds each task record, as built. One retrieval per scripted query; the fork check is the retrieval-score margin and an LLM judge; the recommendation is the higher-ranked option; the trace is a fixed template filled with the pipeline’s sources and recommendation. Every evaluation, fired or not, is logged. Only the rendering of line 6 differs between conditions.

3.4 Why this is an agentic, human-centered study

Tasks are built by the pipeline in Figure 2. For each candidate fork we author two options, the passage that states each, the spec-sheet lines and the keys; the pipeline then decides whether the fork exists and what the assistant recommends. It retrieves the top four chunks for the scripted query (the two authored passages are appended if not already retrieved), and runs the fork check, which fires when the cosine margin between the top two chunks is at most 0.10 *and* an LLM judge, shown the passages, answers that they give different correct answers depending on a fact about the

user that the query does not state, naming at least two options grounded verbatim in the passages. A fork becomes a task only if the judge’s grounded options match the two authored ones; every evaluation is logged, and when a phrasing fails the next neutral phrasing is tried. Over 25 phrasings for 15 candidate forks the check fired on 17 and on the authored fork on 8, from which the six forks of Table 1 were selected. The recommendation is the option whose passage ranks higher for the query; that passage is Source A and the other option’s passage (or, when both options sit in one passage, the page stating the governing rule) is Source B. Each task is then frozen as a JSON record (sheet line, query, the seven trace lines with per-line delays, fork line index, the two options, recommended option, key, the two source pages with their highlighted lines, and the fork-check result), and the web app reads only the record, so every participant sees identical content and the flag changes only the fork line’s rendering.

Pre-generated, replayed behavior. Agent behavior is generated by the pipeline before the sessions and replayed identically to every participant, following the fixed-advice paradigm common in AI-reliance studies [2, 13, 14]^[verify] and the older practice of controlling system behavior in intelligent-interface studies [4]^[verify], so that escalation points, recommendations and their correctness are controlled. The pipeline runs locally on an open-weight model at temperature 0 with a fixed seed; the snapshot, task records, fork-check log and model digests are released. Authors may edit wording but never an escalation decision or a recommendation; recommendation correctness is set by the sheet line, not by overriding the agent. Participants are told the assistant’s responses were prepared in advance. The study thus isolates the human-facing form

of an agent-initiated escalation, not the agent’s behavior or multi-turn interaction.

This passes the “if a chatbot can do it” test in one specific sense: a chatbot answers the query as asked, whereas this pipeline inspects its own retrieval, decides that the answer depends on something it was not told, and hands the decision back. The human-centered question is what that hand-back should look like, and every outcome is a human behavior. We claim no more agency than that. Two properties are the pipeline’s own and are verifiable in the released records: the trigger (the fork decision and the recommendation come from retrieval and the judge, never from us), and the point of the hand-back (it arrives at a fixed step inside a multi-line session rather than after a finished answer). Two are not: the seven-line trace is a template filled with the pipeline’s sources and recommendation, not model-written text, and the task records were built by the single-step pipeline above, not by the planning loop that also exists in the released code and runs only in the playground’s live mode. The system is therefore accurately described as a RAG assistant with a system-initiated escalation at a detected fork, and we use “agentic” in the title only for the hand-back pattern it instantiates.

3.5 Corpus and tasks

The corpus is a frozen snapshot of the CSU CS department systems documentation (sna.cs.colostate.edu), crawled on 3 October 2026: 77 pages (23,965 words), stored as raw HTML and Markdown with a hashed manifest and split at headings into 226 chunks. Every key is derivable from the snapshot plus one sheet line, with no administrator verification; the snapshot and task records are released together; there are 12 measured tasks in three sets of four (job submission; access and remote connection; storage and software) plus 2 practice tasks; and Falcon is a system many CSU students have not used. Each set is two fork source-pairs used twice each, once with the recommended option correct and once wrong, and no source pair recurs across sets, although the sets are not confined to disjoint parts of the site (one source in each of Sets 2 and 3 is a cluster page). Table 1 lists the twelve tasks as built. We keep the FictionalQA blind-versus-informed check [6] at two levels. Over the corpus, the drafting model wrote 111 span-checked questions and answered each blind and with its passage: 84 failed blind and passed informed, 25 were answerable blind, and 2 failed informed. For each candidate fork, one probe question on its deciding fact was answered blind and with the two sources; 13 of 15 forks were answered wrongly blind and correctly informed, and the two that were not were excluded. Task sets are identical by construction in trace length (seven lines) and fork position (line 6); each fork’s exclusivity type (whether the non-key option is invalid under the sheet line or only scoped by the page to the other situation) is recorded and reported per set, timing is checked in the pilot, and task is a random effect in every model. Because someone who already knows the `gpu_short` limit could resolve task J1b without opening a source, we screen for prior use, prefer forks whose condition is stated only on the page, and log what was opened.

No control tasks. The previous draft added two no-fork control tasks per block, for a baseline page-opening rate and so that two citations never signalled a fork. Both purposes are moot here:

Table 1: The twelve measured tasks as built (system/data/tasks). Each set is tied to one form. Each fork is used twice with the same query, trace, sources and recommendation; the twins differ only in the sheet line and question, so that the recommendation is correct under one and wrong under the other.

ID	Sheet line that decides the task	Source A / Source B and the two options	Recommended → key
<i>Set 1 (S1): job submission. Assignment: fine-tune an image model on Falcon in stages.</i>			
J1a	Stage 3, evaluation sweep: about 10 h on 1 GPU.	Partitions, QoS / Policies and Conduct, Resource Usage: <code>gpu_short</code> vs. <code>gpu_medium</code> .	<code>gpu_short</code> (correct)
J1b	Stage 2, training: about 30 h on 1 GPU.	Same.	<code>gpu_short</code> (wrong) → <code>gpu_medium</code> (correct)
J2a	Stage 4, hyper-parameter search: 3 GPUs in one job, about 12 GB each.	GPU Jobs / Resource Limits: <code>kestrel-gpu</code> vs. <code>peregrine-gpu</code> .	<code>kestrel-gpu</code> (correct)
J2b	Stage 1, warm-up training: 1 GPU with about 40 GB.	Same.	<code>kestrel-gpu</code> (wrong) → <code>peregrine-gpu</code>
<i>Set 2 (S2): access and remote connection. Assignment: run the project from a laptop and the CS labs.</i>			
R3a	You are sitting at a CS lab machine.	Using JupyterNotebook (two passages): <code>jupyter-notebook</code> vs. <code>--no-browser</code> <code>--port=9090</code> with an SSH tunnel.	tunnel (wrong) → <code>jupyter-notebook</code>
R3b	You are connected to a CS machine over SSH from your laptop.	Same.	tunnel (correct)
R6a	You are about to SSH to the Falcon cluster.	Virtual Machines / Connecting to Falcon: CSU <code>eName@colostate.edu</code> vs. CS Linux account credentials.	<code>eName</code> (wrong) → CS Linux credentials
R6b	You are about to open the course virtual machine in the VMware portal.	Same.	<code>eName</code> (correct)
<i>Set 3 (S3): storage and software. Assignment: prepare the data and software environment.</i>			
S1a	Stage 5 saves the final 2 GB model, which you must keep.	Temporary Disk Space / Home Directories: <code>/s/<host>/a/tmp</code> vs. <code>home</code> directory.	<code>tmp</code> (wrong) → <code>home</code>
S1b	Stage 1 writes about 40 GB of intermediate files you can regenerate.	Same.	<code>tmp</code> (correct)
S2a	Your login shell is <code>bash</code> (the default).	FAQ / User Accounts: <code>modules.sh</code> vs. <code>modules.csh</code> in the shell start-up file.	<code>modules.sh</code> (correct)
S2b	Your login shell was changed to <code>tcsh</code> last year.	Same.	<code>modules.sh</code> (wrong) → <code>modules.csh</code>

every trace cites Source A and Source B, so two citations carry no signal, and the baseline that matters for H1 is the provenance-only form itself, in which nothing announces the fork. Cutting them is a deliberate simplification: it costs a within-participant estimate of spontaneous source opening with no fork present, which none of H1–H3 needs, and buys a shorter block, a longer trace per task, and one scored field instead of a card of decoy dropdowns. The fork base rate is therefore 100% in every block and is disclosed up front (Section 5).

4 SYSTEM VISION AND TECHNOLOGY

The apparatus is one web application (plain JavaScript, served by a Python standard-library HTTP server) with a condition flag, run in a desktop browser with no headset or spatial component. The screen has the two panes of Figure 1. A replay engine reads the task’s JSON record and prints the trace at a fixed 3–5 s per line; the flag selects one of three renderings of the fork line and nothing else. Clicking a source chip opens the frozen passage in an overlay over

the trace with the relevant lines highlighted; a close control returns to the trace, and under S1 the chips stay active while the modal is up. Every interaction is logged as a timestamped event (block, form, task, Send, each trace line onset, fork onset, modal pick, each source opened and closed, autofill, Accept, Reject, picker choice, final value, block boundaries) to a local SQLite database under a participant code. Instruments are planned as full-screen steps between blocks. The drafting pipeline is Python: the snapshot is chunked by heading, embedded with nomic-embed-text into a vector index, and queried once per scripted query; qwen2.5:7b-instruct (through Ollama 0.35.1, at temperature 0 with a fixed seed) acts as the fork judge and as the blind-versus-informed answerer. Every model call is cached and committed with the model digests, so the records rebuild byte-identically. The pipeline and the web application are built and released in the repository's system/ directory; the between-block instruments are not yet implemented in the application.

5 EVALUATION PLAN

Participants. CSU students who have not used Falcon (screening item), recruited through the class data-collection form; the realistic pool is upper-division undergraduates and new graduate students. $N=18$ target (three per block order), 12 minimum (two per order); pilot of 3–4 not included. The class form suffices for the course; if we intend to publish, an IRB protocol with a faculty principal investigator is needed before recruitment, and we will raise it with the instructor in the first two weeks of October.

Design. Independent variables: (1) Escalation Form, three levels, S1 modal stop, S2 inline warning, S3 provenance-only; within subjects, one level per block. (2) Recommendation Correctness, two levels, correct and wrong, defined per task by whether the recommended option equals the key derived from the spec sheet; within subjects, two tasks per level in every block, in an order fixed per participant by a shuffle seeded from the participant code and set. *Nuisance variable:* Block Order, the six permutations of (S1, S2, S3), between subjects, assigned by the experimenter in rotation from a worksheet; order is the only thing that differs between groups. *Design statement:* a 3×2 within-subjects factorial, Escalation Form $\times$ Recommendation Correctness, two replicates per cell per participant: 12 tasks per participant (6 correct-recommendation, 6 wrong) and, at $N=18$, 216 tasks and 36 per cell. Each form's block carries its own fixed set of four tasks (Table 1). A between-subjects design would need roughly three times as many people for the same power, which at class scale leaves four to six per condition, so we keep within subjects.

Dependent variables, per task unless stated: (a) *Verified*, binary, at least one of the two fork sources viewed for at least 3 s of summed dwell between the start of the task and the final Accept; raw openings and dwell are logged so the threshold can be re-analyzed. (b) *Source pattern*, four-level under the same threshold: the recommended source only (the one cited on the decision line), the non-recommended only, both, or neither; and *Dwell*, seconds each source was open, summed over openings. (c) *Followed*, binary, final value equals the recommended option; crossed with Recommendation Correctness this gives the four cells of appropriate reliance, following a correct recommendation and overriding a

wrong one being appropriate [13], from which relative AI reliance and relative self-reliance follow. (d) *Accuracy*, binary, final value equals key. (e) *Decision time*, seconds from onset of the fork line (or modal) to the final Accept; the scripted preamble is identical within a task and is excluded. (f) Per block: raw NASA-TLX, one perceived-intrusion item (1–7, as in MiPP-Eval, for comparability) and a manipulation check (identify the form just used). (g) Per session: a ranking of the three forms and a free-text reason. Nothing else is collected. Verified is the primary verification measure, for continuity with the binary hypothesis; Source pattern and Dwell are secondary and reported descriptively by form.

Procedure. A session takes about 45 minutes. (1) Consent and instructions, 5 min. The participant reads the cover story: “You are a student using the Falcon cluster for an assigned project. You are given the assignment details and an agentic AI assistant for the cluster documentation that retrieves from those documents. You will complete a series of tasks derived from the assignment.” The instructions state that the assistant is usually right but can be wrong, that the spec sheet describes their assignment, and that they may open any source the assistant cites. (2) Practice, 3 min: two tasks on a practice sheet with no fork; the trace cites two agreeing sources and fills the field, and the experimenter has the participant open and close a source and use Reject and Accept once each. These are the only unforked tasks and are not scored. (3) Three measured blocks, about 6 min each. At the start of a block the sheet for that block's assignment replaces the previous one and stays on screen; the block's four tasks then run in the participant's seeded order through the loop of Section 3.2, and the block ends with raw NASA-TLX, the intrusion item and the manipulation check as full-screen steps. (4) Closing questionnaire, 5 min: the ranking and its reason. Every deciding fact appears on exactly one sheet line and is tested by exactly one task. Block position enters the models as a covariate and the first block is reported alone as a descriptive between-subjects check.

Hypotheses. H1 (verification). The proportion of tasks with $\text{Verified} = 1$ is ordered $S1 > S2 > S3$; the tested contrasts are S1 vs. S2 and S2 vs. S3, each one-sided. Secondary, descriptive: Dwell on the non-recommended source follows the same order. **H2 (appropriate reliance).** On wrong-recommendation tasks the override rate, $P(\text{Followed} = 0)$, is higher under S1 and under S2 than under S3; on correct-recommendation tasks the follow rate, $P(\text{Followed} = 1)$, does not differ across forms. Together these are a Form $\times$ Correctness interaction on Followed, with the form effect confined to wrong-recommendation tasks. **H3 (cost).** Decision time per task and raw NASA-TLX per block are ordered $S1 > S2 > S3$. H1 and H3 together are the trade-off reported for forcing functions [2]; H2 is what matters to a designer. The ranking is reported descriptively.

Analysis. Primary analyses are at the task level (four tasks per form per participant, two per cell): logistic mixed models for Verified and for Followed, with fixed effects of form, correctness, their interaction and block position, and random intercepts for participant and task; a linear mixed model on log decision time; repeated-measures ANOVA with Greenhouse–Geisser correction, or Friedman, for TLX and intrusion. Planned contrasts (S1 vs. S2, S2 vs. S3)

are Holm-corrected and reported as odds ratios or d_z with 95% CIs. Source pattern is reported as a form $\times$ pattern contingency table with Dwell medians. Free-text reasons are coded thematically by both authors. A paired t -test at $N=12$ detects only large participant-level effects ($d_z \approx 0.9$ at 80% power), which is why task-level models are primary; we will simulate power for the H2 interaction from pilot estimates.

Threats. Strategy carry-over (learning that every task hides a fork) is addressed by the up-front instruction, all six orders, the position covariate and the first-block check; the fork base rate is 100% in every block, so carry-over cannot favor one form. Task set and form are confounded by the fixed tie between each form and its four tasks; the construction-level matching, the distinct source pairs and the task random effect are the defenses, and the confound is stated as a limitation. Domain knowledge that resolves a fork without opening a source (including guessing from the flag names) is addressed by screening, by keying on limits stated only on the page, and by logs. Forced choice under S1 is inherent to blocking; verification is never forced, and under S1 the chips stay reachable while the modal is up. Generality is limited to one corpus, novices, a short lab session and replayed single-turn sessions without follow-up questions, and is stated as a scope condition.

6 DELIVERABLES

Built (October): the frozen snapshot (77 pages, hashed manifest); the drafting pipeline with its fork check and a log of every evaluation over 25 query phrasings for 15 candidate forks (17 fired, 8 on the authored fork); the contamination-filter counts (84 of 111 corpus questions kept; 13 of 15 forks); 12 measured and 2 practice task records in three sets of four with their spec sheets; and the web application with the three renderings, replay engine, source overlays and event logging, all released in the repository’s system/directory. **By mid-November:** the between-block instruments; a completed pilot of 3–4 people with tasks and timing revised; and the IRB decision. **By the early-December presentations:** 12–18 completed sessions; the analysis scripts and results for H1–H3; a paper draft with Method and Results; a demo video of one task in all three forms; and the released snapshot plus task records. The critical path is task authoring, not code.

Team. Group CS567_Coen_Cam_Context_Not_Found. Coen Petto: study design, web application and participant sessions. Cameron Suess: drafting pipeline, task authoring and corpus snapshot. Both authors write and analyze.

7 CONCLUSION

We propose a within-subjects study of how an agentic RAG assistant should hand a decision back when the right answer depends on a fact about the user it was never told. The agent, its escalation points and its provenance are held constant; only the form varies, and the recommendation is wrong for this participant on half of the tasks, so verification has a payoff. The prediction is that a modal stop buys the most verification at the highest cost, that an inline warning captures most of the benefit for less, and that a provenance-only citation, which is what current RAG interfaces provide and what MiPP’s C5 counsels, leaves the agent’s errors largely uncaught. Whichever way the results fall, they answer a

question that MiPP, the agentic-RAG pattern and the reliance literature each pose and none has tested.

USE OF AI

Claude Code (Anthropic), running Claude Opus 5.5 with Claude Fable 5.1 subagents, was used in preparing this checkpoint to: critique the instructor’s feedback and the original proposal; read the full text of the MiPP paper and check our claims against it; check each bibliographic entry against publisher records (Crossref, IEEE Xplore, Springer, ACM, ICLR proceedings, arXiv) and produce corrected BibTeX; draft the text of this document; and draw the TikZ mock-ups. A further pass with Claude Code (Claude Fable 5.1) rewrote the draft to this version from our whiteboard notes, which fixed the Falcon -short/-medium example, the spec sheet, the per-task trace and the 12-task design, and redrew the mock-ups to match; the bibliography was not changed in that pass. A further pass with Claude Code (Claude Fable 5.1) corrected this proposal against the built apparatus so that it describes the system as released, including the pipeline, the task set and the verification rule. The authors made every design decision, revise the text, and verify each reference on the publisher page before submission; fields the tool could not confirm are marked with a “verify” comment in the source and omitted until confirmed.

REFERENCES

- [1] Gagan Bansal, Tongshuang Wu, Joyce Zhou, Raymond Fok, Besmira Nushi, Ece Kamar, Marco Tulio Ribeiro, and Daniel S. Weld. 2021. Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems (CHI '21)*. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3411764.3445717>
- [2] Zana Bućinca, Maja Barbara Malaya, and Krzysztof Z. Gajos. 2021. To Trust or to Think: Cognitive Forcing Functions Can Reduce Overreliance on AI in AI-assisted Decision-making. *Proceedings of the ACM on Human-Computer Interaction* 5, CSCW1, Article 188 (2021), 21 pages. <https://doi.org/10.1145/3449287>
- [3] Tathagata Chakraborti, Sarath Sreedharan, Sachin Grover, and Subbarao Kambhampati. 2019. Plan Explanations as Model Reconciliation – An Empirical Study. In *2019 14th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*. IEEE, Daegu, Korea (South), 258–266. <https://doi.org/10.1109/HRI.2019.8673193>
- [4] Nils Dahlbäck, Arne Jönsson, and Lars Ahrenberg. 1993. Wizard of Oz Studies: Why and How. In *Proceedings of the 1st International Conference on Intelligent User Interfaces (IUI '93)*. ACM, New York, NY, USA, 193–200. <https://doi.org/10.1145/169891.169968>
- [5] Eric Horvitz. 1999. Principles of Mixed-Initiative User Interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '99)*. ACM, New York, NY, USA, 159–166. <https://doi.org/10.1145/302979.303030>
- [6] John Kirchenbauer, Natjanaan Mongkolsupawan, Yuxin Wen, Tom Goldstein, and Daphne Ippolito. 2026. FictionalQA: A Dataset for Studying Memorization and Knowledge Acquisition. In *The Fourteenth International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=SUNC1eJGMr>
- [7] Daniel C. McFarlane. 2002. Comparison of Four Primary Methods for Coordinating the Interruption of People in Human-Computer Interaction. *Human-Computer Interaction* 17, 1 (2002), 63–139. https://doi.org/10.1207/S15327051HCI1701_2
- [8] Daniel C. McFarlane and Kara A. Latorella. 2002. The Scope and Importance of Human Interruption in Human-Computer Interaction Design. *Human-Computer Interaction* 17, 1 (2002), 1–61. https://doi.org/10.1207/S15327051HCI1701_1
- [9] Saroj Mishra, Suman Niroula, Umesh Yadav, Dilip Thakur, Srijan Gyawali, and Shiva Gaire. 2026. SoK: Agentic Retrieval-Augmented Generation (RAG): Taxonomy, Architectures, Evaluation, and Research Directions. arXiv:2603.07379 [cs.AI] <https://arxiv.org/abs/2603.07379> arXiv preprint, 7 March 2026; no published version found as of September 2026.
- [10] Martin H. Möhle, Manuel Zall, Tim Thorwart-Gumpert, Nico Elbert, Frédéric Thiesse, and Christoph M. Flath. 2026. “Trust me, Bro!” On the Influence of Agentic Support on Trust Calibration in AI-Assisted Decision Making. In *Artificial Intelligence in HCI: 7th International Conference, AI-HCI 2026, Held as Part of*

- the 28th HCI International Conference, HCII 2026, Proceedings, Part II, Helmut Degen and Stavroula Ntoa (Eds.). Lecture Notes in Computer Science, Vol. 16744. Springer, Cham, 472–488. https://doi.org/10.1007/978-3-032-30860-3_30
- [11] Divya Ravi and Renuka Sindhgatta. 2025. Exploring Trust and Transparency in Retrieval-Augmented Generation for Domain Experts. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3706599.3719985>
 - [12] Luis Martín Sánchez-Adame and Sonia Mendoza. 2026. MiPP: Minimal Proactivity Policies for Timely, Low-Intrusion Proactivity. In *Artificial Intelligence in HCI: 7th International Conference, AI-HCI 2026, Held as Part of the 28th HCI International Conference, HCII 2026, Proceedings, Part II*, Helmut Degen and Stavroula Ntoa (Eds.). Lecture Notes in Computer Science, Vol. 16744. Springer, Cham, 363–382. https://doi.org/10.1007/978-3-032-30860-3_23
 - [13] Max Schemmer, Niklas Kuehl, Carina Benz, Andrea Bartos, and Gerhard Satzger. 2023. Appropriate Reliance on AI Advice: Conceptualization and the Effect of Explanations. In *Proceedings of the 28th International Conference on Intelligent User Interfaces (IUI '23)*. ACM, New York, NY, USA, 410–422. <https://doi.org/10.1145/3581641.3584066>
 - [14] Helena Vasconcelos, Matthew Jörke, Madeleine Grunde-McLaughlin, Tobias Gerstenberg, Michael S. Bernstein, and Ranjay Krishna. 2023. Explanations Can Reduce Overreliance on AI Systems During Decision-Making. *Proceedings of the ACM on Human-Computer Interaction* 7, CSCW1, Article 129 (2023), 38 pages. <https://doi.org/10.1145/3579605>